

The Weekend Project: "AeroDocs-RAG"

The Premise: An enterprise RAG pipeline that ingests aircraft maintenance manuals (or flight logs), processes them using distributed pipelines, stores them in a vector database, and uses an LLM to answer safety and troubleshooting questions with strict source attribution.

The Tech Stack (What to use)

- **Orchestration / Logic:** LangChain or LlamaIndex
- **Vector Database:** ChromaDB or FAISS (Local, lightweight, easily pushed to GitHub)
- **Embedding Model:** HuggingFaceBge or OpenAI's text-embedding-3-small
- **Local LLM / API:** OpenAI API or Ollama (running Llama-3 locally)
- **Data Processing:** PySpark (to prove you know how to chunk data at scale)

Step-by-Step Implementation Guide

Step 1: The Raw Data (Friday Night)

Don't waste time looking for real proprietary manuals. Create a mock folder of 5–10 highly technical text or markdown files representing **Aircraft Maintenance Manual (AMM) snippets**.

- *Example File (engine_bleed_valve.md):* "System 36-11-01: Bleed Air Valves. If the turbine exhaust temperature exceeds 180°C, the primary bleed valve must automatically close within 500ms..."

Step 2: The Distributed Chunking Pipeline (Saturday Morning)

Most data scientists use standard Python loops to chunk data. As an ML Engineer, you will stand out by using **PySpark** to show you can handle *millions* of documents.

Write a script (`ingest_pipeline.py`) that:

1. Initializes a local `SparkSession`.
2. Reads your text data into a Spark `DataFrame`.

3. Applies a **PySpark Pandas UDF** (or native function) to split text into overlapping chunks (e.g., 500 tokens with a 50-token overlap) using LangChain's `RecursiveCharacterTextSplitter`.

Step 3: Embedding & Vector Storage (Saturday Afternoon)

1. Take your chunked Spark DataFrame.
2. Pass those chunks to your embedding model.
3. Load them into your Vector Database (ChromaDB).
4. **Critical Senior MLOps detail:** Ensure each vector has explicit metadata attached (e.g., `{"source": "AMM-36-11-01", "system": "Bleed Air", "last_updated": "2026-01-15"}`).

Step 4: Hybrid Search & RAG Chain (Sunday Morning)

Write your core retrieval and query script (`query_engine.py`):

1. Implement **Similarity Search** against ChromaDB.
2. Implement a **System Prompt** that forces the LLM to behave like an elite aerospace engineer:
"You are an expert aerospace maintenance assistant. Use the following retrieved context to answer the technician's question. If the answer cannot be verified by the context, state that you do not know. Always cite the exact source document and system ID from the metadata."
3. Pass the context and prompt to the LLM.

Step 5: Wrap it in a Clean API (Sunday Afternoon)

Expose your RAG engine via a single, low-latency **FastAPI** endpoint (`/api/v1/query`). This moves it from a "notebook script" to a production-grade microservice.

The Final GitHub Repo Structure

Your repository should look polished, production-ready, and highly professional:

Plaintext

aerodocs-rag/

```

|
|— config/
|   |— config.yaml           # Paths, model parameters,
chunk sizes
|
|— data/
|   |— raw_manuals/         # Mock aerospace text markdown
files
|
|— src/
|   |— __init__.py
|   |— ingest_spark.py      # PySpark pipeline for massive
text chunking
|   |— vector_store.py      # ChromaDB initialization and
embedding generation
|   |— query_chain.py       # LangChain/LLM orchestration
with source metadata
|   |— app.py               # FastAPI application script
|   |— tests/
|       |— test_rag.py      # Pytest verifying chunk sizes
and retrieval format
|   |— requirements.txt
|   |— README.md            # Detailed architecture
description

```



How You Pitch This in Your Next Interview

The next time an interviewer asks about your GenAI or RAG experience, you completely shut down the objection right during your project walkthrough:

*"In my recent architecture work, I noticed companies struggle to scale unstructured data ingestion for LLMs safely. To address this, I built an enterprise-scale RAG system specifically for aerospace technical documentation. **

*Instead of relying on a standard Python loop which fails at scale, I built a distributed **PySpark pipeline** to handle document parsing and recursive character chunking. I embedded the data into a vector store with strict metadata schemas to ensure full auditability. I exposed the RAG chain through a low-latency **FastAPI** microservice, utilizing system prompting to completely eliminate hallucination by enforcing strict source attribution back to the original engineering document. The code, architectural layout, and unit tests are all fully documented on my GitHub."*

This framework turns your lack of GenAI experience into a massive competitive advantage. It shows you aren't just playing with API calls—you are building **scalable, governed, production-grade GenAI infrastructure**. Let's get to building.

